

Partial-Evaluation Templates: Accelerating Partial Evaluation with Pre-compiled Templates

Florian Huemer
Johannes Kepler University, Linz, Austria
florian.huemer@jku.at

Aleksandar Prokopec
Oracle Labs, Zurich, Switzerland
aleksandar.prokopec@oracle.com

David Leopoldseder
Oracle Labs, Vienna, Austria
david.leopoldseder@oracle.com

Raphael Mosaner
Oracle Labs, Linz, Austria
raphael.m.mosaner@oracle.com

Hanspeter Mössenböck
Johannes Kepler University, Linz, Austria
hanspeter.moessenboeck@jku.at

Abstract—Partial evaluation allows specializing programs by substituting some of their inputs with static values and evaluating the resulting static expressions. When applied to a language interpreter, it specializes the interpreter to a fixed source program, resulting in a specialized interpreter that existing compilers can optimize and generate efficient machine code for. Although this approach achieves good peak performance in just-in-time compilers, partial evaluation is costly with respect to compilation time. In the case of bytecode interpreters, partial evaluation must process and transform each bytecode instruction. Since instructions with the same opcodes share common functionality, partial evaluation applies the same transformations repeatedly. Consequently, partial evaluation significantly increases the compilation time of just-in-time compilation.

We propose partial-evaluation templates, i.e., reusable collections of ahead-of-time-compiled graphs that are generated by a hand-written cogen approach. The templates are parametric in their static inputs, and cover common functionality in a bytecode interpreter implementation. At compile time, templates specialize themselves by substituting their static inputs, reducing themselves to a single pre-compiled graph that the compiler inlines and optimizes. This reduces the size of the intermediate representation and speeds up partial evaluation, as well as optimizations.

We integrated our approach into GraalVM, a state-of-the-art high-performance polyglot virtual machine, and GraalWasm, a bytecode-interpreter-based WebAssembly runtime. We extended the existing GraalVM compiler with a binding-time analysis that generates the templates and introduced a specialized for the templates into the existing partial evaluator. We enabled template generation for nearly all opcodes in GraalWasm, which reduced partial-evaluation time by up to 36 % and warmup time by up to 17 % without impacting peak performance.

Index Terms—partial evaluation, binding-time analysis, cogen, templates, GraalVM, virtual machines, escape analysis

I. INTRODUCTION

Partial evaluation (PE) is an optimization technique that allows specializing programs by substituting some of their inputs with static values and evaluating the resulting static expressions [1]–[3]. When applied to a language interpreter, a partial evaluator can specialize the interpreter to a fixed source program, which results in a target program that only covers the execution logic found in the source program [1]–[4]. This approach is often referred to as the 1st Futamura projection [4]. If the target program is in an intermediate representation (IR)

that is supported by an existing virtual machine (VM), the included just-in-time (JIT) compiler can optimize the target program and generate efficient machine code for it [5], [6].

Although PE in JIT compilers achieves peak performance close to custom compiler implementations [6]–[8], it can significantly increase compilation time. In specific compiler configurations, the partial-evaluation time can account for more than half of the total compilation time [9], [10].

In the case of bytecode interpreters, PE must iterate over all instructions and process various interpreter methods for them [11]. Instructions with the same opcodes share common functionality, apart from statically known differences, such as the stack slots to read or the bit width of operations. Therefore, PE applies the same transformations and inlining decisions numerous times. Consequently, this leads to duplication in the target program, such as duplicated nodes in a graph-based IR.

Duplication negatively impacts the performance of compiler optimizations at run time. JIT compilers often apply optimizations, such as escape analysis [12], [13] or read elimination [14], after PE. The execution time of these optimizations typically increases with the size of the IR, for example, the number of nodes in a graph-based IR. Therefore, it is crucial to minimize the IR size to keep overall compilation time low.

To illustrate these problems, consider the hypothetical implementation of a stack-machine-based bytecode interpreter. Fig. 1 shows the interpreter’s *store* instruction, which writes an integer value to memory. Method *store* takes parameters *f*, representing the expression stack, *sp*, the stack pointer, and *mem*, a memory representation. The method loads the address and value from the stack and calls *storeInt* on the memory.

In this example, we assume that PE processes two consecutive *store* instructions, and the parts of the source producing the required addresses and values. In addition, we assume that the target program is represented by a graph-based IR.

Fig. 2 shows a subgraph of the target program. In the graph, solid lines represent control-flow edges pointing down to successors, while dashed lines represent usages pointing up to inputs. Nodes 1 and 2 represent parameters *f* and *mem*. Since we assume that *sp* is static in our example, and PE evaluates static expressions at compile time, PE replaces the

```

1 void store(Frame f, int sp, Mem mem) {
2   int adr = f.getInt(sp - 1);
3   int val = f.getInt(sp - 2);
4   mem.storeInt(adr, val);
5 }

```

Fig. 1. Implementation of the *store* method, writing an integer value to memory, in a stack-machine-based bytecode interpreter.

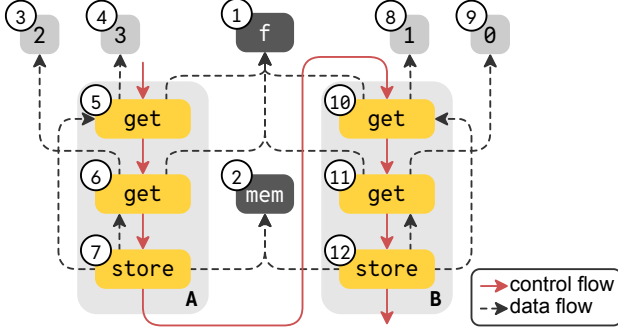


Fig. 2. Identical subgraphs of two *store* instructions in the target program.

expressions $sp - 1$ and $sp - 2$ in lines 2 and 3 of Fig. 1 with constant nodes. This results in nodes 3 and 4 for the first *store* instruction and nodes 8 and 9 for the second one. PE transforms the *getInt* calls in lines 2 and 3 into *get* nodes 5 and 6 for the first *store* instruction and nodes 10 and 11 for the second one. It further transforms the *storeInt* call in line 4 into *store* nodes 7 and 12.

Apart from potentially different static parameters and constants computed from them, PE produces identical subgraphs for both *store* instructions. Fig. 2 reflects this by the two highlighted subgraphs A and B. If a compiler applies optimizations after PE, they would visit the same three nodes in both subgraphs and potentially apply identical transformations.

Therefore, we propose to extend the 1st Futamura projection with reusable templates that are generated ahead of time by a custom binding-time analysis [2], [15] of a hand-written cogen approach [3], [16]. Templates are represented by single nodes in the IR of target programs, as shown in Fig. 3, and support compiler optimizations. Each template is a collection of specialized IR graphs (*residual graphs*) that are parametric in their static inputs (question marks in Fig. 3). At compile time, the templates specialize themselves by selecting one of the residual graphs and substituting the static inputs with constants. This eliminates repetitive work and reduces the impact on optimizations. Instead of fully replacing the 1st Futamura projection by the cogen approach [16], our approach focuses on specific interpreter parts to keep the memory overhead low [10]. The contributions of this paper are therefore:

- We present *partial-evaluation templates*, i.e., collections of residual graphs for frequently executed methods of bytecode-interpreter implementations, which are generated by a hand-written cogen approach and specialize themselves at compile time (Section III).

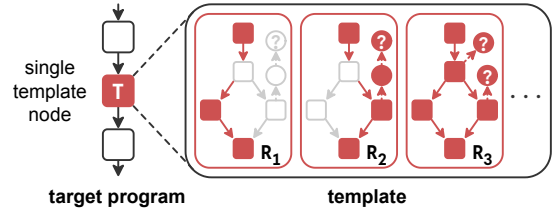


Fig. 3. A partial-evaluation template represented by a single template node in the target program. Each template contains multiple residual graphs.

- We describe a use-sensitive and context-sensitive binding-time analysis for a Sea-of-Nodes-based IR that performs control-flow duplication, constant folding, and partial-evaluation-template generation (Section III-A).
- We present the integration of partial-evaluation templates into existing compiler optimizations, as well as the introduction of *template escape analysis* (Section IV).
- We show that the integration of our approach into GraalVM, a state-of-the-art high-performance virtual machine, and GraalWasm, a bytecode-interpreter-based WebAssembly runtime, reduces partial-evaluation time by up to 36 % and warmup time by up to 17 % (Section V).

II. BACKGROUND

We describe partial-evaluation templates as a general approach to reduce the partial-evaluation time of the 1st Futamura projection and apply it to a specific VM to demonstrate its applicability. Therefore, we begin our technical content by describing PE and the VM we worked with, GraalVM.

A. Partial Evaluation

Partial evaluation, also called *mixed computation* [17], is a specialization technique that takes a source program *source*, combined with parts of its inputs *in1*, to construct a new, faster target program *target*. When given the remaining inputs *in2*, *target* produces the same result that *source* produces when provided with both *in1* and *in2* [1], [2], [18].

$$\llbracket \text{source} \rrbracket [\text{in1}, \text{in2}] = \llbracket \text{target} \rrbracket [\text{in2}]$$

To produce *target*, a partial evaluator, often called *mix* [17], *precomputes* all expressions at partial-evaluation time that only depend on *in1*, *inlines* [19] function calls, and *reduces* remaining expressions such as $x \times 1$ to x . The values known at PE time *in1* and expressions that can be computed from them are called *static*, while the remaining values *in2* and expressions on them are called *dynamic* [1], [2].

PE approaches are categorized as either *online* or *offline* [2]. Online approaches are typically one-phase processes that perform the analysis of whether a value is static or dynamic, and the specialization simultaneously. Offline approaches split the analysis and specialization into two phases. First, a *binding-time analysis* annotates all operations in *source*, such as expressions and parameters, as either static or dynamic. Then, a *specializer* specializes *source* to a set of static inputs based on the given annotations [2], [18].

Partial evaluation can generate *specialized interpreters* (target programs), *compilers*, and *compiler generators* in the context of language runtimes. The relevant PE approaches are referred to as the Futamura projections [4].

1) *First Futamura Projection*: In the 1st Futamura projection, `mix` specializes an interpreter `int` to a fixed source program `source`, resulting in a faster specialized interpreter called `target` [4].

```
target = [[mix]] [int,source]
```

Since `int` and `source` are fixed during this process, `mix` removes the parts of the interpreter responsible for syntax and semantic analysis, leaving only the execution logic found in `source` [4]. Depending on the output language of the PE process, `target` can either be directly executed, or further optimized by an optimizing compiler [5], [6].

2) *Second Futamura Projection*: The 2nd Futamura projection uses self-application. Here, `mix` specializes itself to a fixed `int`, resulting in a `compiler`. When provided with `source`, the `compiler` produces a `target` program [2].

```
compiler = [[mix]] [mix,int]
target = [[compiler]] [source]
```

3) *Third Futamura Projection*: The 3rd Futamura projection, often called the *cogen approach* [2], [20], can be achieved through the same idea of self-application, also called *autoprojection* [18]. Here, `mix` specializes itself to itself, resulting in a compiler generator `cogen`. When provided with `int`, `cogen` produces a `compiler`, often more generally called program generator [2], or *generating extension* [17].

```
cogen = [[mix]] [mix,mix]
compiler = [[cogen]] [int]
```

Practical self-application is often hard to achieve, resulting in a limited set of supported language features or only modest speedups [16], [20]. Birkedal and Welinder [16] showed that hand-writing `cogen` with the help of a modified *binding-time analysis* is as easy as writing `mix`. In addition, a hand-written `cogen`-approach addresses many of the issues of self-application in statically typed languages [16]. Hand-written `cogen` approaches and most self-application approaches require an offline partial evaluator [2].

B. GraalVM

GraalVM [5] is a state-of-the-art high-performance Java Virtual Machine (JVM) [21], which includes a dynamic optimizing compiler called *GraalVM compiler*. The *GraalVM* compiler supports JVM languages, is written in Java [22], and supports ahead-of-time [23] and just-in-time [5] compilation. When using JIT compilation, programs start in the interpreter and the *GraalVM* compiler compiles frequently executed methods [24]. The *GraalVM* compiler applies numerous optimizations [25] and makes extensive use of *speculative optimizations* [24] based on *assumptions* [26]. If an assumption no longer holds, *GraalVM* invalidates the generated machine code associated

with the assumption and transfers execution back to the interpreter [6], [26] through *deoptimization* [27].

The *GraalVM* compiler uses *Graal IR* [24], [28], a Sea-of-Nodes-based [29], [30] intermediate representation, on which it performs all optimizations. This SSA-based [31], [32] IR models control-flow and data-flow dependencies between nodes. It differentiates between *fixed nodes*, which have control-flow dependencies that define their execution order, and *floating nodes*, which only have data-flow dependencies. In Fig. 2, the *get*, *load*, and *store* nodes represent fixed nodes, since they include implicit control flow in the form of null checks required by Java [22], while the parameters and constants represent floating nodes.

When deoptimizing, *GraalVM* translates the machine state, i.e., registers and native stack frames, back to the interpreter state, i.e., JVM stack frames [26]. Furthermore, it re-allocates objects that were scalar-replaced by escape analysis [24]. To enable this translation, *Graal IR* contains *frame-state nodes*, a mapping of run-time values to corresponding JVM-stack-frame slots. Initially, frame-state nodes are associated with points in the graph where side effects may occur and the compiler moves them to actual deoptimization points later in the compilation process. Side effects are operations that might not lead to the same results when re-executed, such as field stores or method calls [26]. In Fig. 2, each *store* node would have an associated frame-state node.

In addition to JVM languages, *GraalVM* supports the execution of guest languages through partial evaluation [6] and provides a domain-specific language for defining interpreters via the *Truffle framework* [7]. *Truffle* represents an abstraction layer in *GraalVM* and uses Java annotations to aid in interpreter design and PE. *Truffle* primarily focuses on AST interpreters [7], but also supports bytecode interpreters by implementing a bytecode loop inside a flat AST structure [33]. In this case, one of the AST nodes contains a bytecode loop.

GraalWasm [34] uses the *Truffle framework* to implement a bytecode-interpreter-based WebAssembly [35] runtime on top of *GraalVM*. It implements the required flat AST structure by using two nodes per WebAssembly function, the root node and a so-called *function node* that contains all the bytecode-interpreter logic, including the bytecode loop [36].

C. GraalVM Partial Evaluation

GraalVM implements an online partial evaluator that takes a *Graal-IR* representation of an interpreter and specializes it to an AST, which represents the static source program. During PE, the address and type of each AST node is considered constant. This enables devirtualization of virtual method calls inside the interpreter, replacing indirect with direct calls, and *constant folding* [37] of interpreter values [6]. PE starts at the first node in the *Graal-IR* representation of the interpreter and processes all fixed nodes in reverse post order. It iterates all inputs of fixed nodes and processes floating nodes along the way based on their usages. PE applies several optimizations.

PE inlines direct calls by processing them recursively instead of emitting call instructions. This eliminates all method calls

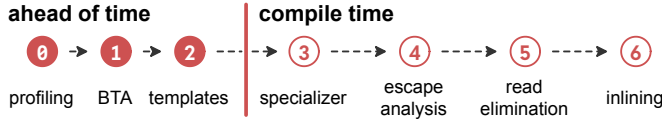


Fig. 4. Overview of the generation and usage of partial-evaluation templates.

in the interpreter, such as the call to `storeInt` in line 4 of Fig. 1. PE furthermore intrinsifies calls to compiler-intrinsic methods by replacing them with special nodes in the IR. In Fig. 1, this applies to the `f.getInt` calls in lines 2 and 3, which PE replaces with `get` nodes in Graal IR.

The `Frame` object represents the local state of a guest language function, such as local variables or the expression stack in a bytecode interpreter. PE intrinsifies accesses to the `Frame` to enable escape analysis, which allocates the local state on the native stack or in registers.

PE replaces field and array-element loads by constants if the object’s address is constant and the accessed field or array element is considered immutable. A field is immutable if it has the Java `final` modifier, or is annotated with the Truffle `@CompilationFinal` annotation. This annotation states that a field can be modified during interpretation but is considered constant during PE. If a field that has an object type is immutable, the object’s address is considered constant, while for primitive types, the value is considered constant. PE replaces arithmetic and logic operations with constants if all their inputs are constants and performs dead code elimination to eliminate resulting unreachable code paths [6].

Truffle provides an `@ExplodeLoop` [38] annotation to specify that the outermost loop of an annotated method in the interpreter implementation must be fully unrolled. When PE encounters this annotation, it duplicates the body of the loop for each loop iteration until the entire loop is removed. This allows PE to transform a bytecode interpreter loop into a linear sequence of instructions in Graal IR [33].

The GraalVM compiler applies several optimizations on the Graal-IR graph of the target program. This includes escape analysis [26], read elimination, and conditional elimination.

III. PARTIAL-EVALUATION TEMPLATES

Our approach to *ahead-of-time template generation* (ATG) is a hand-written cogen approach that creates program generators called *partial-evaluation templates*. To generate the templates, as shown on the left side of Fig. 4, ATG gathers profiling information and uses an offline partial-evaluation approach to perform a binding-time analysis (BTA) on a Sea-of-Nodes-based IR. Instead of specializing the entire bytecode interpreter, ATG takes individual methods in the bytecode interpreter and specializes them ahead of time into partial-evaluation templates based on the profiling information.

During compilation, a *specializer* replaces missing values in the templates, and the compiler further optimizes the partial-evaluation templates via escape analysis and read elimination, before inlining the templates into the IR. Fig. 4 shows the steps involved in this process on the right side.

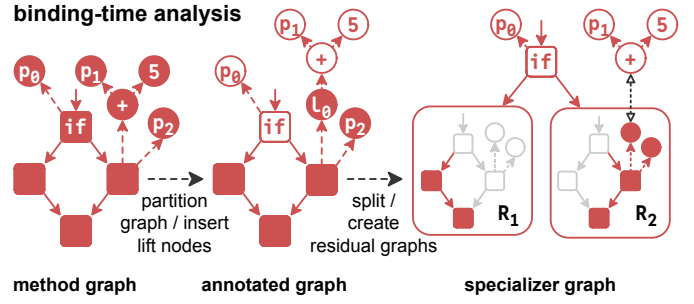


Fig. 5. Binding-time analysis in ATG, generating multiple residual graphs in a *specializer graph*.

A. Binding-Time Analysis

In a Sea-of-Nodes-based IR, every node represents a literal, control flow, or an operation that defines a value based on its inputs [29]. In the context of PE, each of these nodes can be either executed at partial-evaluation time, or at run time. Therefore, the BTA must annotate each node as either static or dynamic, thereby partition the graph into two subgraphs, a *specializer graph* for static nodes, and a *residual graph* for dynamic nodes. All *specializer graphs* and *residual graphs* of the same method form a *partial-evaluation template*.

The proposed binding-time analysis in ATG identifies a node as static if one of the following conditions holds:

- The node is a *constant node*. Constant nodes represent literals, such as integers or floating-point numbers.
- The node is a *parameter node* and annotated as static. Parameter nodes represent the formal parameters of a method. The annotation as static can either be manual, e.g., via annotations in the interpreter source code, or automatically derived, e.g., via profiling information as described in Section III-C.
- All inputs of the node are static and the node represents an idempotent, side-effect-free, non-trapping operation. For example, a division is not static in most cases as a division by 0 would lead to a runtime error.

Fig. 5 shows the binding-time analysis in ATG. It starts with the graph representation of the method it should analyze, called the *method graph*, and partitions the graph into the static and dynamic subgraphs in the *annotated graph*. Filled nodes in the graphs represent dynamic nodes, while not filled nodes represent static nodes. In Fig. 5, p_0 , p_1 , and p_2 represent parameter nodes. Node 5 represents a constant node for the integer value 5, while $+$ represents an addition node.

The BTA starts at the first fixed node in the graph and follows all control-flow edges in reverse post-order. In Fig. 5 fixed nodes are square, while floating nodes are circular. Before processing a fixed node, the BTA transitively follows its input edges to find floating nodes that are not annotated yet, and annotates them according to the defined conditions. In Fig. 5, BTA starts at node *if* and first processes p_0 . Since we assume for this example that p_0 and p_1 are manually annotated as static, the BTA annotates nodes p_0 , p_1 , 5, $+$, and *if* as static.

When the BTA reaches the end of the graph, it moves all static nodes into the specialized graph, while the dynamic nodes remain in the annotated graph. At this point, the annotated graph only contains operations that are executed at run time and it is now called the residual graph.

Our BTA approach has the following additional properties:

1) *Flow Insensitive*: An analysis is *flow sensitive* when it allows for a variable to have a different binding time at different program points, i.e., it can be static at one point and dynamic at another [15]. Sea of Nodes builds a form of SSA at parse time [29], and each variable is assigned exactly once. This allows our analysis to be flow insensitive while still producing a flow-sensitive result [15], [39], [40].

2) *Control-Flow Splitting and Duplication*: Our BTA performs control-flow splitting and duplication based on static control flow, i.e., control-flow nodes that are annotated as static by the BTA. As soon as the BTA reaches a control-flow node that is annotated as static, and its condition is not a constant node, the BTA duplicates the annotated graph into multiple graphs depending on the number of successor branches of the control-flow node. In each individual annotated graph, the BTA replaces the condition of the control-flow node with a constant node such that in each of the graphs, the control flow continues in a distinct branch. Furthermore, it moves the control-flow node, and all nodes relevant for computing its condition, into the specialized graph. Each of the successor branches in the specialized graph points to its associated annotated graph. The BTA processes all resulting annotated graphs recursively.

In Fig. 5, the BTA duplicates the annotated graph into two separate graphs R_1 and R_2 based on the *if* node. In R_1 , the BTA replaces node p_0 by `true` and in R_2 , it replaces node p_0 by `false`. Due to the dead code elimination performed by the BTA, only the *then* branch (left branch) remains in R_1 , while in R_2 , only the *else* branch (right branch) remains. The BTA moves nodes *if* and p_0 into the specialized graph and links the successor edges to R_1 and R_2 .

In addition to splitting of static control flow, the BTA can also introduce *inline caches* [41] for virtual calls. If the receiver of a virtual call is a static node, i.e., it will be replaced by a constant node with a fixed type at partial-evaluation time, the BTA introduces an inline cache into the specialized graph and splits the annotated graph based on the possible subtypes of the receiver type. In the resulting annotated graphs, the virtual calls are devirtualized based on the inline cache, which allows for additional inlining.

To avoid a state explosion in the specialized graph, the BTA can use heuristics or profiling information to only introduce splits if it results in a significant reduction of the graph size of annotated graphs or additional optimization potential [42]. Furthermore, the BTA can introduce fallback mechanisms for infrequent branches into the specialized graph that fall back to the 1st Futamura projection at partial-evaluation time [10]. The fall back inlines the method and recursively partially evaluates it, as is usually performed for interpreter methods in GraalVM.

This form of control-flow splitting does not support loops, since it would require significant engineering effort to get the

existing online partial evaluator in GraalVM and our proposed specialized (Section III-B) to collaborate on loop unrolling. Therefore, loops always remain in the residual graphs.

3) *Context Sensitive*: An analysis is *context sensitive* [15], often called *polyvariant* [18], [43], if it can analyze a method with respect to different combinations of binding-time annotations for the parameters of the method. The different binding-time annotations may arise when the method is called from different locations in a program with different calling contexts.

To support context sensitivity, our BTA inserts an *annotation-switch node* at the top of each specialized graph. This node branches based on the binding-time annotations of parameters in the method signature. Each distinct set of binding-time annotations leads to a different annotated graph, similar to control-flow splitting. The BTA specializes the resulting annotated graphs based on the specific binding-time annotations. Furthermore, the annotation-switch node has a default branch that indicates that no residual graph exists for the given set of binding times and uses a fall back at partial-evaluation time.

4) *Use Sensitive*: An analysis is *use sensitive* if it allows for the result of an operation to be used in both a static and a dynamic context [15], [44]. In ATG this means that the values represented by nodes in the specialized graph are also available in the residual graphs. Making a static value available in a dynamic context is often called *lifting* [44].

To support this operation, we make the transition between specialized graphs and residual graphs explicit via so called *lift nodes*. Our BTA inserts a lift node every time a node that is non-idempotent, has side effects, or can trap at runtime, has an input node that is annotated as static. The lift nodes represent a form of parameter node in the residual graphs. Constant nodes do not require lift nodes and are duplicated instead.

In Fig. 5, the *else* branch of the *if* node (right branch) uses node $+$. If we assume that the *else* branch has side effects and $+$ is annotated as static, the BTA inserts lift node l_0 to make it explicit that the value of $+$ is required in the residual graphs.

5) *Optimizations*: Since the BTA is based on an existing online partial evaluator, it applies its optimizations while annotating the graph. Those include constant folding, global value numbering [45], dead code elimination, and inlining.

B. Specializer

We extended the existing online partial evaluator in GraalVM with a custom *specializer*. When PE encounters a call to a method at partial-evaluation time for which a partial-evaluation template exists, the specializer substitutes all parameters in the specialized graph of the template with the arguments of the call, and reduces the specialized graph to a single residual graph, as shown in Fig. 6. Here, the specializer replaces p_0 in Fig. 5 with `false`, represented by node f , and p_1 with the constant node 4. The specializer eliminates the *then* branch of the *if* condition, resulting in the residual graph R_2 , and reduces the addition of nodes 4 and 5 to the constant node 9.

The specializer furthermore replaces the call node representing the method call by a *template node*, represented by T in Fig. 6. The template node references the selected residual

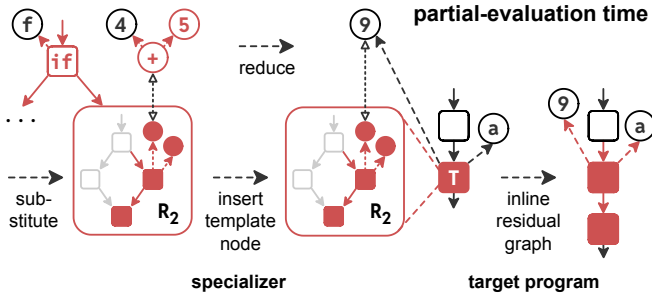


Fig. 6. ATG specializer, processing the partial-evaluation template generated in Fig. 5, substituting parameters by constants, reducing the specializer graph to a single residual graph, and inserting a template node into the target program.

```

1 @Template
2 void store(Frame f, int sp, Mem mem, int bits) {
3   int adr = f.getInt(sp - 1);
4   if (bits == 32) {
5     int val = f.getInt(sp - 2);
6     mem.storeInt(adr, val);
7   } else {
8     long val = f.getLong(sp - 2);
9     mem.storeLong(adr, val);
10  }
11 }

```

Fig. 7. Extension of the *store* method in Fig. 1, allowing for 32-bit (int) and 64-bit (long) memory writes based on the *bits* parameter.

graph, all arguments of the call that remain after the specializer reduces the specializer graph, and all constant nodes required by lift nodes inside the residual graph, such as node 9 in Fig. 6.

After it applies escape analysis and read elimination, as explained in Section IV, the compiler inlines the template into the target program, i.e., it replaces the template node with the nodes inside the residual graph. Furthermore, the compiler replaces parameter nodes and lift nodes of the residual graph with nodes referenced by the template node, as shown in Fig. 6.

By reusing residual graphs, PE at compile time no longer needs to recursively process methods in the interpreter implementation for which templates exist. This reduces the time spent for partial evaluation and subsequent optimization phases. Furthermore, since ATG represents each template by a single node, partial-evaluation templates reduce the size of the IR.

C. Profiling

ATG requires a way to identify bytecode interpreter methods it should transform into templates. In our implementation, we chose an explicit approach based on a `@Template` Truffle annotation to give interpreter developers control over which methods become templates. Alternatively, a profiling run could provide call frequencies for the methods and a threshold could define, for which of the methods to generate templates.

In addition, ATG requires information about the calling contexts of the methods to extract possible binding-time annotations for their parameters. We profile the existing online partial evaluator in GraalVM before ATG to extract the relevant binding times for each calling context. Furthermore, we use a binary path profile [46] to identify paths encountered during online PE in annotated methods. We use the identified paths to aid in control-flow splitting and duplication during ATG.

To demonstrate this approach, Fig. 7 extends our introductory example. In comparison to Fig. 1, the *store* method has a `@Template` annotation and has a new parameter *bits*, which determines, whether the store method should write a 32-bit (int) or 64-bit (long) integer value to memory.

In our example, *store* is encountered three times during the profiling, resulting in the paths in Fig. 8. The first time, the arguments are `[Frame f, const 2, const MemA mem, const 32]`, where **const** represents a constant node.

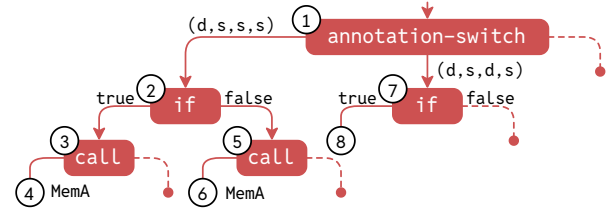


Fig. 8. Binary path profile of the method in Fig. 7.

Each formal parameter for which a constant node argument exists is annotated as static (s), while all other parameters are annotated as dynamic (d). Since all arguments except for the frame are constant nodes, the first calling context has the binding-time annotations (d, s, s, s) . Similar to the specializer graph, the binary path profile in Fig. 8 starts with an annotation switch for the different binding-time annotations. Since *bits* is 32 in the calling context, and *mem* has the concrete type MemA, the profile captures the path 1–2–3–4. In Fig. 8, node 2 represent the condition in line 4 of Fig. 7, while node 3 represent the inline cache for the virtual call in line 6.

When PE encounters *store* for the second and third time, the arguments in our example are `[Frame f, const 4, const MemA mem, const 64]`, leading to path 1–2–5–6, and `[Frame f, const 4, Mem mem, const 32]`, leading to path 1–7–8. In this example, PE did not encounter the dashed paths in Fig. 8, which leads to the generation of fall backs in the specializer graph during ATG.

IV. COMPILER OPTIMIZATIONS

In addition to reducing partial-evaluation time and IR size, ATG can reduce the execution time of compiler optimizations by pre-computing relevant information inside residual graphs. For example, since the BTA in ATG visits every node in a method graph ahead of time to annotate the graph, it can also identify sets of nodes relevant for a specific optimization. The read elimination performed by the GraalVM compiler only considers *load* and *store* nodes. Therefore, the BTA can build up a set of *load* and *store* nodes while annotating the graph and provide it to the *read-elimination* phase at compile time.

Alternatively, ATG can apply optimizations to residual graphs after the BTA to produce multiple versions of a residual graph

depending on the parameters of the method. For example, ATG could produce one residual-graph version where it fully unrolls a loop, and a second version, where the loop is not unrolled. At compile time, a *loop-unrolling phase* could then choose between one of the two versions.

We integrated partial-evaluation templates into two compiler optimizations in GraalVM. We used the first approach of building up a set of nodes during the BTA for our implementation of read elimination, and the second approach of building multiple versions of residual graphs for our implementation of escape analysis. Since the read-elimination implementation only required minor modifications of the existing approach in GraalVM, the rest of this section focuses on escape analysis.

A. Partial Escape Analysis

Escape analysis allows compilers to identify whether an object *escapes* its allocating method, e.g., whether it is passed as an argument to another method or assigned to a global variable. If this is not the case, a compiler can eliminate the object allocation and replace the object fields with local variables, which is called *scalar replacement*. It furthermore allows compilers to eliminate reads and writes to non-escaping objects, by mapping reads to the values of preceding writes [26].

GraalVM uses *partial escape analysis* [26] (PEA) as its escape analysis implementation. PEA starts at the first fixed node in the graph and follows the control-flow edges. If PEA encounters an allocation node, e.g., an allocation of a new object or an array with a fixed length at compile time, it replaces the allocation node by a *virtual object*, an abstract representation of the object that would be allocated at run time. PEA uses virtual objects to simulate the effects of reads and writes during escape analysis. Virtual objects contain *slots* for all fields or array elements of the object. Each slot is a reference to a node in the graph. When PEA encounters a *store* node to a field or compile-time-constant array index of a virtual object, it replaces the associated slot in the virtual object by the node representing the new value. Furthermore, PEA removes the *store* node, since the object associated with the store no longer exists at run time. When PEA encounters a *load* node from a field or compile-time-constant array index of a virtual object, it replaces the *load* node with the value in the associated slot of the virtual object. Afterwards, PEA removes the *load* node. As a result, reads are mapped to the values of preceding writes and PEA scalar replaces the object.

When PEA encounters a node that leads to the escape of the object, e.g., a call node that uses the object as an argument, or a store node that accesses the object at a non-constant array index, PEA introduces a new allocation node for the object. At this point, the allocation node uses the current slots in the virtual object to initialize the escaped object. The processes of allocating an escaped object is called *materialization* in PEA.

GraalVM furthermore requires virtual objects for deoptimization and attaches *virtual-object nodes*, representing the slots of the virtual object, to frame-state nodes. When a deoptimization happens, the JVM uses the virtual-object slots to allocate the object required by the Java interpreter.

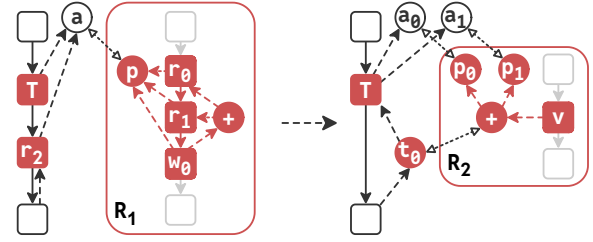


Fig. 9. Template escape analysis replacing object parameters with parameters for individual fields or array elements, and object writes with return values.

The existing PEA implementation in GraalVM evaluates template nodes similar to call nodes, since it cannot reason about the contained residual graphs. Therefore, PEA always materializes all virtual-object parameters of a template node before the template. This leads to additional object allocations, which reduces the performance of the compiled code and furthermore increases the IR size, since load, store, and allocation nodes remain in the IR. To address this issue, partial-evaluation templates must be able to receive scalarized objects as inputs and must communicate writes involving object parameters inside the residual graphs to PEA at run time.

B. Template Escape Analysis

We propose *template escape analysis* (TEA), an ahead-of-time escape-analysis approach based on PEA. TEA scalar replaces the parameters and return values of methods and uses *effect lists* to build multiple *versions* of residual graphs.

TEA analyzes reads inside residual graphs to identify fields or array elements (*members*) of object parameters required inside a method. For each required member, TEA introduces a new parameter and replaces reads to the member with references to this parameter. Fig. 9 shows the transformation of residual graph R_1 into R_2 . R_1 uses object parameter p corresponding to object a in the target program, and reads two members from p , represented by read nodes r_0 and r_1 . TEA replaces r_0 and r_1 with the new parameters p_0 and p_1 , and removes the original parameter p , as well as r_0 and r_1 , in R_2 . The addition node $+$, using r_0 and r_1 as inputs in R_1 , uses p_0 and p_1 in R_2 . Parameters p_0 and p_1 correspond to individual members a_0 and a_1 of a in the target program.

TEA furthermore analyzes writes inside residual graphs to identify members of object parameters updated inside the method. For each updated member, TEA captures the new value by a *return-value* node and removes the writes. In Fig. 9, w_0 writes $+$ to a member in p . This value is read by r_2 in the target program. TEA removes w_0 and captures $+$ with return-value node v . At compile time, PEA introduces *template-value* node t_0 into the target-program graph. Node t_0 corresponds to $+$ captured by v . Furthermore, PEA replaces the read of r_2 by t_0 . TEA generates an effect list to allow the compiler to extract a_0 and a_1 from a at compile time and to introduce t_0 .

As a result of TEA scalar replacing parameters in residual graphs, PEA does not materialize a before the template node at compile time and passes its members a_0 and a_1 instead.

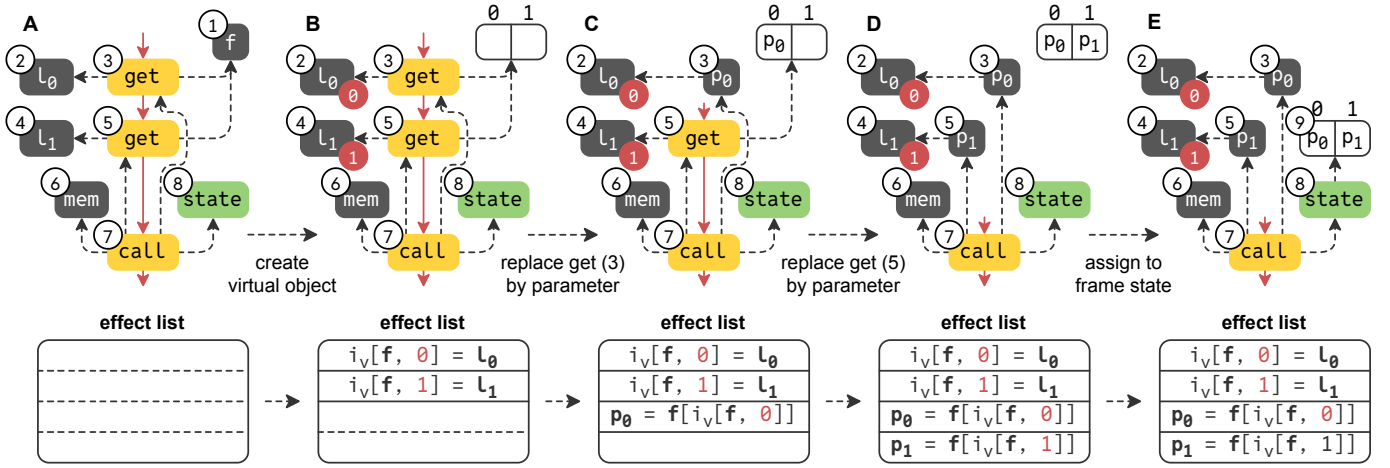


Fig. 10. Template escape analysis removing frame accesses (get nodes) in the residual graph and replacing them with parameters nodes.

Similar to the binding-time annotations of parameters in the BTA of ATG, TEA specializes the residual graph based on *escape annotations*, i.e., whether an object parameter is virtual or materialized. At run time, PEA selects one of the residual-graph versions based on these escape annotations.

TEA builds a residual-graph version for each combination of escape annotations. For a method with one object parameter, TEA would build two versions. One version, where the parameter is virtual, and one version, where the parameter is materialized. For multiple parameters, this implementation needs to consider a possible state-explosion problem, since the number of residual-graph versions corresponds to the size of the powerset of all object parameters. This can be addressed by limiting the number of considered parameters or to only create versions for escape annotations encountered in a profiling run.

To show the creation of a residual-graph version, we use the *store* method in Fig. 7. In our example, we apply TEA to the residual graph produced by the BTA for path 1–7–8 in Fig. 8. Graph A in Fig. 10 shows this graph. Node 1 represents parameter f and node 6 parameter mem . Node 2 is the lift node for the expression $sp - 1$ in line 3 and node 4 for $sp - 2$ in line 5, while nodes 3 and 5 represent their corresponding $f.getInt$ operations. Node 7 represents the call to $mem.storeInt$ in line 6, while node 8 is a frame-state node associated with the call. In this example, TEA builds the residual-graph version for the escape annotations $(v, -, m, -)$. TEA only considers dynamic parameters for escape analysis. Parameter f is virtual (v) and mem is materialized (m) in our example. The static parameters are represented by a hyphen.

In addition to PEA, TEA also considers virtual parameters during escape analysis and represents them by virtual objects, similar to allocation nodes. While the number of fields of objects are statically known in Java, array lengths are not, especially when arrays are passed as parameters to methods. Therefore, TEA does not know the number of slots for all virtual objects of parameters. Furthermore, compile-time-constant array indices for accesses of virtual objects in residual graphs are either represented by constant nodes or lift nodes.

While TEA can access constant-node values ahead of time, the values of lift nodes are only available at compile time.

Consequently, TEA first iterates over the graph to identify the size of virtual objects, and to assign *virtual indices* i_v to the constant nodes and lift nodes used by array accesses. Each object gets a distinct set of virtual indices. Since the frame f in Fig. 10 is an array-like data structure, the lift nodes l_0 and l_1 used by *get* nodes 3 and 5 are array indices. TEA assigns $i_{v,f} = 0$ to l_0 and $i_{v,f} = 1$ to l_1 , as shown by the circles right to l_0 and l_1 in graph B of Fig. 10. For PEA to resolve the actual array indices of l_0 and l_1 at compile time, TEA makes an entry for each virtual index in the effect list of the residual graph, as shown below graph B in Fig. 10.

Due to global value numbering performed by the BTA of ATG, residual graphs represent static expressions computing the same value by the same lift node. For example, if $sp - 1$ would be computed a second time in the *store* method in Fig. 7, the residual graph would represent both computations by l_0 . Therefore, TEA can correctly map reads and writes to the same array index. Our implementation also considers *array-index aliasing*, i.e., different computations that result in the same array index. However, this is omitted from this paper.

Since TEA identifies two distinct virtual indices for accesses to f in Fig. 10, TEA replaces parameter f with a virtual object with two slots, as shown in graph B. At the start of TEA, these slots reference a special node representing the *empty* value.

After TEA transforms all virtual parameters into virtual objects, it performs identical operations to PEA but uses virtual indices instead of actual array indices. When TEA encounters a read operation of a slot that still references the *empty* value, as is the case for slot 0 accessed by *get* node 3 in Fig. 10, it replaces the *get* node by a new parameter node. Furthermore, TEA creates an entry in the effect list to be able to resolve the value of the new parameter at compile time. Fig. 10 shows the updated graph and the additional effect-list entry in graph C. The effect-list entry defines that PEA can resolve p_0 based on virtual object f and virtual index 0. Additionally, TEA adds p_0 into slot 0 of the virtual object, so subsequent accesses to

virtual index 0 can reuse the parameter. TEA repeats the same process for *get* node 5 as shown in graph D of Fig. 10.

When TEA encounters a node with an attached frame-state node, such as node 8 in Fig. 10, it creates a virtual-object node and adds it as an input to the frame-state node. In Fig. 10, graph E introduces virtual-object node 9 into the graph.

When TEA reaches the end of the graph, it iterates the slots inside virtual objects to find newly written values. If a slot is not empty and does not contain a parameter, TEA introduces a return-value node that references the value inside the slot. Furthermore, TEA creates an effect-list entry for the creation of a new template-value node at run time.

V. EVALUATION

To show the effectiveness of partial-evaluation templates, we evaluated their impact on PE time and compilation time. We evaluated their effect on application warmup and made sure that they do not impact peak performance or code size. Furthermore, we evaluated the reduction of the target-program IR size, the fall-back frequency, the profiling overhead, the template generation time, and the memory usage of PE templates.

We conducted our evaluation by enabling partial-evaluation templates in GraalWasm [36]. Since GraalWasm represents its bytecode interpreter logic inside a single method [47], we extracted the logic of individual opcodes into separate methods in the interpreter and annotated them with `@Template`.

A. Benchmarks and Setup

We used 14 benchmarks from different sources, including JetStream 2 [48], an internal benchmark suite, and standalone applications such as *jpeg* [49], *ogg* [50], and the *photon* [51] image-processing library for our evaluation. The source code of the internal benchmark suite is publicly available as part of the GraalWasm repository [52]. The benchmarks execute real-world computing tasks, including image-, video- and audio processing, simulation of event-based systems, sorting algorithms, and the Fast Fourier transform [53]. All benchmarks are written in C, C++, or Rust, and are compiled with WASI SDK [54] version 25 or version 1.84.1 of the Rust compiler.

We executed all benchmarks on the Community Edition of GraalVM (commit: 54b5f72b [55]) and ran the benchmarks on an Intel Core i5-8365U with 8 logical cores at a fixed CPU frequency of 2.30 GHz and turbo boost disabled. The system has 16 GB of main memory and runs Fedora Linux 41 (Workstation Edition). We compiled GraalVM and GraalWasm with *LabsJDK CE 24* (labsjdk-ce-24+12-jvmci-b01) [56].

The compilation thresholds in Truffle, i.e., how often a function must be executed in the interpreter before it is considered for compilation, are highly tuned to the GraalVM compiler implementation before the addition of partial-evaluation templates. Therefore, a reduction of the compilation time of individual functions due to partial-evaluation templates impacts the compilation behavior and the order in which functions are scheduled for compilation. Since we want to obtain comparable numbers by compiling the same methods in the same order between GraalVM before (*baseline*) and

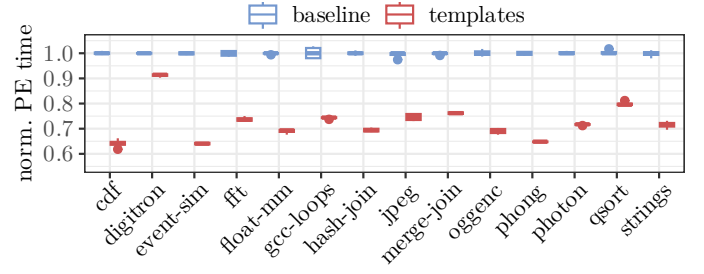


Fig. 11. Normalized partial-evaluation time in GraalWasm benchmarks with templates disabled (baseline) and enabled (templates); lower is better.

after partial-evaluation templates, we disabled *background compilation* for the GraalVM compiler when measuring partial-evaluation and compilation time. This has the effect that compilation runs interleaved with the benchmark in the same thread. As soon as a function in the benchmark should be compiled, the GraalVM compiler takes over the thread and compiles the function. When it is finished, the thread continues in the benchmark.

We ensured that the GraalVM compiler compiles all relevant WebAssembly functions in each benchmark by using sufficient warmup iterations. We used cross validation [57] to not overfit our templates to the current benchmark. Hence, for the measurement of each benchmark, we used the respective 13 other benchmarks for profiling. We ran each benchmark 4 times in separate processes to account for measurement inaccuracies due to garbage collection and non-deterministic optimization phases in the compiler. We normalized all results to the median of the baseline.

B. Measurements

1) *PE Time*: Fig. 11 shows the partial-evaluation time with and without templates. The average time is reduced between 20 % to 35 % for most benchmarks, with a maximum reduction of 36 % in the *cdf* and *event-sim* benchmarks. The *digitron* benchmark shows the least improvement with a reduction of only 9 %. This benchmark contains a lot of indirect calls and guest language control flow, which we currently cannot represent with templates, as explained in Section VI.

2) *Compilation Time*: Fig. 12 shows the compilation time of escape analysis, read elimination, and template inlining performed after partial evaluation with and without templates. While some benchmarks, such as *digitron*, *fft*, *merge-join*, and *qsort*, do not show improvements, most benchmarks show a time reduction between 4 % and 9 %. In the four benchmarks without any effect, the time reduction in escape analysis is negated by the additional cost for template inlining, while in all other benchmarks, the cost for template inlining is outweighed by the gains in escape analysis and read elimination.

3) *IR Size*: Fig. 13 shows the node count in Graal IR after online PE with and without templates. The node counts show similar results to partial-evaluation time with reductions between 15 % and 41 % for most benchmarks. The *digitron* benchmark, again, shows the least reduction in node count with only 7 %, caused by indirect calls and control flow.

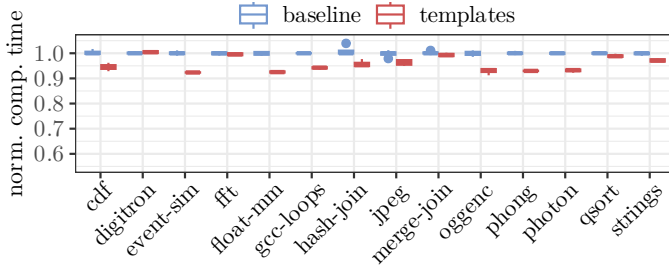


Fig. 12. Normalized compilation time of escape analysis, read elimination, and template inlining after partial evaluation in GraalWasm benchmarks with templates disabled (baseline) and enabled (templates); lower is better.

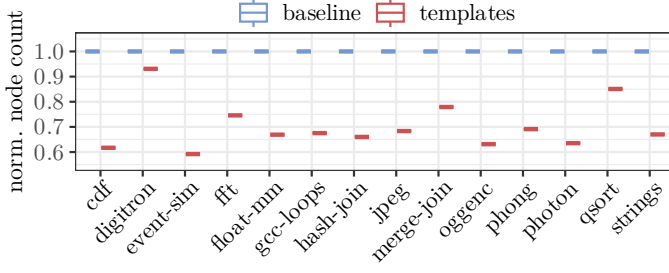


Fig. 13. Normalized node count after PE in GraalWasm benchmarks with templates disabled (baseline) and enabled (templates); lower is better.

4) *Warmup*: In order to get warmup numbers which reflect real world behavior, we re-enabled background compilation for our warmup measurements. Fig. 14 shows the time spent in the 5 first iterations of the *event-sim*, *jpeg*, *oggenc*, and *photon* benchmarks. We selected the benchmarks since they have the largest total partial-evaluation time. In the *event-sim* benchmark, only the first iteration shows an improvement, which results in a warmup-time reduction across all iterations from 22.7 s to 19.4 s (14.5 %). In the *jpeg*, *oggenc*, and *photon* benchmarks, also the time of the second and third iterations are reduced, resulting in warmup-time reductions across all iterations from 8.1 s to 6.7 s (17.3 %) for *jpeg*, 43.3 s to 36.4 s (15.9 %) for *oggenc*, and 81.5 s to 74.6 s (8.5 %) for *photon*. All other benchmarks (not in Fig. 14) show a similar behavior with the main impact in the first iteration of each benchmark.

5) *Peak Performance and Code Size*: We evaluated the peak performance, in operations per seconds, and code size, in bytes, for all 14 benchmarks. The evaluation shows no significant changes, as shown in Fig. 15 and Fig. 16.

6) *Additional Metrics*: We measured the *fall-back frequency*, i.e., the number of times the specialized falls back to the 1st Futamura projection divided by the total number of times PE encounters a method with an associated template. The fall-back frequency across all benchmark is 0.0497 %, while PE does not perform any fall backs in most benchmarks.

We furthermore measured the *profiling overhead*, the additional time required by the online partial evaluator, when profiling is enabled. Profiling includes identifying the binding times of parameters and extracting path profiles. We found an average profiling overhead of 0.0036 % across all benchmarks.

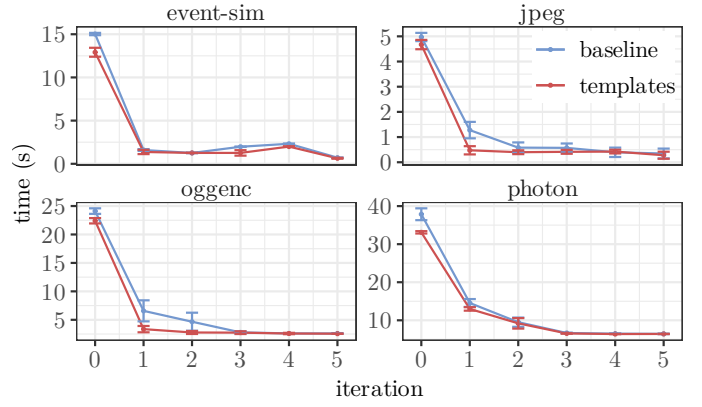


Fig. 14. Warmup of 4 GraalWasm benchmarks with the largest total PE time with templates disabled (baseline) and enabled (templates); lower is better.

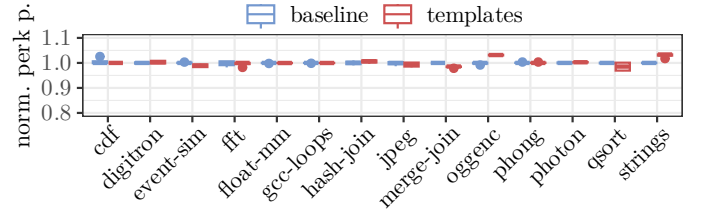


Fig. 15. Normalized peak performance of the GraalWasm benchmarks with templates disabled (baseline) and enabled (templates); higher is better.

In addition, we measured the *template generation time* in μ s and the *memory usage* of templates in bytes. The average time for generating all templates for a benchmark run is 0.68 s of the overall compilation time of 182.25 s, while on average, the benchmarks require 16.27 MB to store the templates.

Complementary metrics are available online [58].

VI. LIMITATIONS

There are two main limitations of partial-evaluation templates in the context of GraalVM. These prevented us from representing two categories of WebAssembly instructions by templates, *calls* and *control flow*.

Guest language calls, e.g., *call* and *call_indirect* instructions in the WebAssembly bytecode, pass their parameters as a Java object array in Truffle. Therefore, *frame* values must be copied into this object array before a call. Due to the design of the *frame* in Truffle, guest language values must be copied one by one in a loop. PEA relies on this loop being unrolled

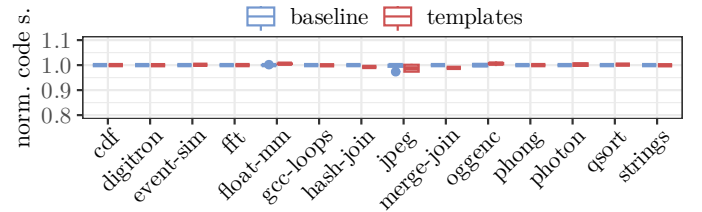


Fig. 16. Normalized compiled code size of the GraalWasm benchmarks with templates disabled (baseline) and enabled (templates); lower is better.

by PE, so that each copy operation uses a constant array index when accessing the *frame*. Since loops always end up in the residual graph of partial-evaluation templates in our approach, the unrolling of these loops is no longer guaranteed and the frame would be materialized during PEA.

PE in GraalVM performs loop reconstruction, i.e., it identifies guest language loops based on control flow. For this to work, the effects of all control-flow branches, such as changes to program counters or stack pointers, must be visible to PE and cannot be inside templates. This includes WebAssembly control-flow instructions, such as `br`, `br_if`, or `if-else`.

Supporting this functionality inside templates would require coordinated loop unrolling between the online partial evaluator in GraalVM and our proposed specialized. We plan to add support for loops in a future version of ATG.

VII. RELATED WORK

There exists a broad spectrum of research on the theoretical and practical basis of partial evaluation and the cogen approach [1], [3], [4], [17], [59], [60]. While most approaches focus on self-application to generate compiler generators, several approaches use a hand-written cogen approach [10], [16], [20], [61], [62], similar to ATG. Among these approaches, most perform their binding-time analysis on a source-code level [16], [20], [61], [62], in contrast to the graph-based intermediate representation in ATG.

Similarly, the research field of binding-time analysis has been explored in various directions, resulting in several binding-time analyses [15], [44], [63], [64] based on abstract interpretation, data-flow analysis, type inference, and constraint solving. Most approaches work on source-code level and combine different BTA properties, such as polyvariance [63], flow sensitivity [15], or use sensitivity [44]. Ponisio [40] proposes a monovariant (as opposed to polyvariant), flow-sensitive binding-time analysis for an SSA-based subset of Java. In contrast to ATG, Ponisio uses constraint solving for their binding-time analysis approach. Grant et al. [65] propose a polyvariant binding-time analysis that performs control-flow splitting based on the binding-times of values at control-flow points, called polyvariant division in their approach. Their work requires explicit annotations to define values as static in an interpreter, compared to the profiling-based approach in ATG. Das et al. [39] propose flow-sensitive binding-time analyses based on a program representation graph (PRG), a program representation that contains some of the aspects of SSA. PRG differs in many aspects from the Sea-of-Nodes-based IR in ATG and their work focuses on building a semantic foundation for BTA instead of a hand-written cogen approach.

Among hand-written cogen approaches and binding-time analyses, the work by Latifi et al. [10], which also builds on the partial evaluation work by Würthinger et al. [6], is closest to our approach. Latifi et al. [10] propose CompGen, an approach loosely based on the second Futamura projection with the goal of reducing partial-evaluation time. Their approach performs a binding-time analysis in a hand-written cogen approach based on Graal IR. In contrast to ATG, CompGen is focused on

AST interpreters in Truffle and generates compiler generators for individual AST nodes as Java source code. The compiler generators produced by CompGen generate Graal-IR graphs at partial-evaluation time, compared to the use of pre-compiled Graal-IR graphs by ATG. Latifi et al. [10] integrated their approach into GraalVM, similar to partial-evaluation templates. However, since their approach focuses on AST interpreters, and ATG focuses on bytecode interpreters, a quantitative comparison would not produce meaningful results.

There exist many approaches [66]–[70] using templates for various compiler optimizations. To the best of our knowledge, no existing approach uses PE to generate templates.

In terms of design and functionality of templates, the approach by Xu and Kjolstad [71] is closest to our work. Xu and Kjolstad [71], propose copy-and-patch, a fast compilation technique with the goal of reducing compilation time. Their template-based compilation approach generates a library of templates with patchable holes, called stencils, ahead of time. The stencils exist in different variants in the library and represent specializations of per-built implementations of AST nodes or bytecode opcodes, similar to the template specializations in partial-evaluation templates. At compile time, their approach selects the appropriate stencil, and patches missing values, similar to the specialized in ATG. Compared to ATG, their template generation does not use partial evaluation.

Lastly, using partial-evaluation templates during compilation is similar to function outlining [72], [73]. Both approaches reduce the size of the function IR to simplify and speed up optimizations. Function outlining explicitly searches for cold parts of a function that can be outlined and are typically not inlined again in the compilation process. In contrast, ATG transforms frequently executed methods, which stay outlined only for a part of the compilation process.

VIII. CONCLUSION

We presented partial-evaluation templates, collections of residual graphs used to speed up the 1st Futamura projection. The templates are generated by a custom BTA for Sea-of-Nodes-based IRs that partitions the IR into specialized graphs and residual graphs. We furthermore showed that by implementing a flow-insensitive, context-sensitive, and use-sensitive BTA that supports control-flow splitting and various optimizations, our approach can produce multiple optimized and simplified residual graphs. In addition, we extended the existing partial evaluator in GraalVM with a specialized that selects a residual graph at partial-evaluation time and inserts template nodes into the target program, instead of performing recursive partial evaluation of interpreter methods, to reduce IR size and compilation time. Through the adaptation of read elimination and the introduction of template escape analysis, template nodes can participate in compiler optimizations without negative impacts on peak performance or code quality. Lastly, we evaluated our approach in GraalWasm, a WebAssembly runtime based on GraalVM, and achieved partial-evaluation-time reductions of up to 36 %, IR-size reductions of up to 41 %, and warmup-time reductions of up to 17 % without impacting peak performance.

REFERENCES

- [1] N. D. Jones, “An introduction to partial evaluation,” *ACM Comput. Surv.*, vol. 28, no. 3, p. 480–503, Sep. 1996. [Online]. Available: <https://doi.org/10.1145/243439.243447>
- [2] P. S. Neil D Jones, Carsten K Gomard, *Partial Evaluation and Automatic Program Generation*. Peter Sestoft, 1993.
- [3] L. Beckman, A. Haraldson, Östen Oskarsson, and E. Sandewall, “A partial evaluator, and its use as a programming tool,” *Artificial Intelligence*, vol. 7, no. 4, pp. 319–357, 1976. [Online]. Available: [https://doi.org/10.1016/0004-3702\(76\)90011-4](https://doi.org/10.1016/0004-3702(76)90011-4)
- [4] Y. Futamura, “Partial evaluation of computation process - an approach to a compiler-compiler,” *High. Order Symb. Comput.*, vol. 12, no. 4, pp. 381–391, Dec 1999. [Online]. Available: <https://doi.org/10.1023/A:1010095604496>
- [5] T. Würthinger, C. Wimmer, A. Wöß, L. Stadler, G. Duboscq, C. Humer, G. Richards, D. Simon, and M. Wolczko, “One VM to rule them all,” in *ACM Symposium on New Ideas in Programming and Reflections on Software, Onward! 2013, part of SPLASH '13, Indianapolis, IN, USA, October 26–31, 2013*, ser. Onward! 2013, A. L. Hosking, P. T. Eugster, and R. Hirschfeld, Eds. New York, NY, USA: ACM, 2013, pp. 187–204. [Online]. Available: <https://doi.org/10.1145/2509578.2509581>
- [6] T. Würthinger, C. Wimmer, C. Humer, A. Wöß, L. Stadler, C. Seaton, G. Duboscq, D. Simon, and M. Grimmer, “Practical partial evaluation for high-performance dynamic language runtimes,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI 2017, Barcelona, Spain, June 18–23, 2017*, ser. PLDI 2017, A. Cohen and M. T. Vechev, Eds. New York, NY, USA: ACM, 2017, pp. 662–676. [Online]. Available: <https://doi.org/10.1145/3062341.3062381>
- [7] C. Humer, C. Wimmer, C. Wirth, A. Wöß, and T. Würthinger, “A domain-specific language for building self-optimizing AST interpreters,” in *Generative Programming: Concepts and Experiences, GPCE'14, Vasteras, Sweden, September 15–16, 2014*, U. P. Schultz and M. Flatt, Eds., vol. 50. New York, NY, USA: ACM, sep 2014, pp. 123–132. [Online]. Available: <https://doi.org/10.1145/2658761.2658776>
- [8] M. Rigger, M. Grimmer, C. Wimmer, T. Würthinger, and H. Mössenböck, “Bringing low-level languages to the jvm: efficient execution of llvm ir on truffle,” in *Proceedings of the 8th International Workshop on Virtual Machines and Intermediate Languages*, ser. VMIL 2016. New York, NY, USA: Association for Computing Machinery, 2016, p. 6–15. [Online]. Available: <https://doi.org/10.1145/2998415.2998416>
- [9] F. Huemer, D. Leopoldsdeder, A. Prokopec, R. Mosaner, and H. Mössenböck, “Taking a Closer Look: An Outlier-Driven Approach to Compilation-Time Optimization,” in *38th European Conference on Object-Oriented Programming (ECOOP 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), J. Aldrich and G. Salvaneschi, Eds., vol. 313. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 20:1–20:28. [Online]. Available: <https://doi.org/10.4230/LIPIcs.ECOOP.2024.20>
- [10] F. Latifi, D. Leopoldsdeder, C. Wimmer, and H. Mössenböck, “Compgen: generation of fast jit compilers in a multi-language vm,” in *Proceedings of the 17th ACM SIGPLAN International Symposium on Dynamic Languages*, ser. DLS 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 35–47. [Online]. Available: <https://doi.org/10.1145/3486602.3486930>
- [11] O. Larose, S. Kaleba, H. Burchell, and S. Marr, “Ast vs. bytecode: Interpreters in the age of meta-compilation,” *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA2, Oct. 2023. [Online]. Available: <https://doi.org/10.1145/3622808>
- [12] Y. G. Park and B. Goldberg, “Escape analysis on lists,” *SIGPLAN Not.*, vol. 27, no. 7, p. 116–127, Jul. 1992. [Online]. Available: <https://doi.org/10.1145/143103.143125>
- [13] B. Blanchet, “Escape analysis for object-oriented languages: application to java,” *SIGPLAN Not.*, vol. 34, no. 10, p. 20–34, Oct. 1999. [Online]. Available: <https://doi.org/10.1145/320385.320387>
- [14] M. Fernández, R. Espasa, and S. Debray, “Load redundancy elimination on executable code,” in *Euro-Par 2001 Parallel Processing*, R. Sakellariou, J. Gurd, L. Freeman, and J. Keane, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 221–229. [Online]. Available: https://doi.org/10.1007/3-540-44681-8_33
- [15] L. Hornof and J. Noyé, “Accurate binding-time analysis for imperative languages: flow, context, and return sensitivity,” in *Proceedings of the 1997 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*, ser. PEPM '97. New York, NY, USA: Association for Computing Machinery, 1997, p. 63–73. [Online]. Available: <https://doi.org/10.1145/258993.259005>
- [16] L. Birkedal and M. Welinder, “Hand-writing program generator generators,” in *Programming Language Implementation and Logic Programming*, M. Hermenegildo and J. Penjam, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1994, pp. 198–214. [Online]. Available: https://doi.org/10.1007/3-540-58402-1_15
- [17] A. Ershov, “Mixed computation: potential applications and problems for study,” *Theoretical Computer Science*, vol. 18, no. 1, pp. 41–67, 1982. [Online]. Available: [https://doi.org/10.1016/0304-3975\(82\)90111-6](https://doi.org/10.1016/0304-3975(82)90111-6)
- [18] A. Bondorf, “Automatic autoprojection of higher order recursive equations,” *Science of Computer Programming*, vol. 17, no. 1, pp. 3–34, 1991. [Online]. Available: [https://doi.org/10.1016/0167-6423\(91\)90035-V](https://doi.org/10.1016/0167-6423(91)90035-V)
- [19] R. W. Scheifler, “An analysis of inline substitution for a structured programming language,” *Commun. ACM*, vol. 20, no. 9, p. 647–654, sep 1977. [Online]. Available: <https://doi.org/10.1145/359810.359830>
- [20] M. Leuschel, J. Jørgensen, W. Vanhoof, and M. Bruynooghe, “Offline specialisation in prolog using a hand-written compiler generator,” *Theory Pract. Log. Program.*, vol. 4, no. 2, p. 139–191, Jan. 2004. [Online]. Available: <https://doi.org/10.1017/S1471068403001662>
- [21] T. Lindholm, F. Yellin, G. Bracha, A. Buckley, and D. Smith, “The java® virtual machine specification,” 2025. [Online]. Available: <https://docs.oracle.com/javase/specs/jvms/se24/jvms24.pdf>
- [22] J. Gosling, B. Joy, G. Steele, G. Bracha, A. Buckley, D. Smith, and G. Bierman, “The java® language specification,” 2025. [Online]. Available: <https://docs.oracle.com/javase/specs/jls/se24/jls24.pdf>
- [23] C. Wimmer, C. Stancu, P. Hofer, V. Jovanovic, P. Wögerer, P. B. Kessler, O. Pliss, and T. Würthinger, “Initialize once, start fast: application initialization at build time,” *Proc. ACM Program. Lang.*, vol. 3, no. OOPSLA, pp. 184:1–184:29, oct 2019. [Online]. Available: <https://doi.org/10.1145/3360610>
- [24] G. Duboscq, T. Würthinger, L. Stadler, C. Wimmer, D. Simon, and H. Mössenböck, “An intermediate representation for speculative optimizations in a dynamic compiler,” in *VMIL@SPLASH '13: Proceedings of the 7th ACM workshop on Virtual machines and intermediate languages, Indianapolis, IN, USA, 28 October 2013*, ser. VMIL '13, C. Bockisch, M. Haupt, S. Blackburn, H. Rajan, and J. Gil, Eds. New York, NY, USA: ACM, 2013, pp. 1–10. [Online]. Available: <https://doi.org/10.1145/2542142.2542143>
- [25] J. Eisl, “Trace register allocation,” 2018. [Online]. Available: <https://resolver.obvsg.at/urn:nbn:at:at-ubl:1-25787>
- [26] L. Stadler, T. Würthinger, and H. Mössenböck, “Partial escape analysis and scalar replacement for java,” in *Proceedings of Annual IEEE/ACM International Symposium on Code Generation and Optimization*, ser. CGO '14. New York, NY, USA: Association for Computing Machinery, 2014, p. 165–174. [Online]. Available: <https://doi.org/10.1145/2581122.2544157>
- [27] U. Hölzle, C. Chambers, and D. M. Ungar, “Debugging optimized code with dynamic deoptimization,” in *Proceedings of the ACM SIGPLAN'92 Conference on Programming Language Design and Implementation (PLDI), San Francisco, California, USA, June 17–19, 1992*, S. I. Feldman and R. L. Wexelblat, Eds., vol. 27. New York, NY, USA: ACM, jul 1992, pp. 32–43. [Online]. Available: <https://doi.org/10.1145/143095.143114>
- [28] G. Duboscq, L. Stadler, T. Würthinger, D. Simon, C. Wimmer, and H. Mössenböck, “Graal ir: An extensible declarative intermediate representation,” in *Proceedings of the Asia-Pacific Programming Languages and Compilers Workshop*, 2013, pp. 1–9. [Online]. Available: https://ssw.jku.at/General/Staff/GD/APPLC-2013-paper_12.pdf
- [29] C. Click and M. Paleczny, “A simple graph-based intermediate representation,” in *Proceedings ACM SIGPLAN Workshop on Intermediate Representations (IR'95), San Francisco, CA, USA, January 22, 1995*, ser. IR '95, M. D. Ernst, Ed. New York, NY, USA: ACM, 1995, pp. 35–49. [Online]. Available: <https://doi.org/10.1145/202529.202534>
- [30] C. Click and K. D. Cooper, “Combining analyses, combining optimizations,” *ACM Trans. Program. Lang. Syst.*, vol. 17, no. 2, pp. 181–196, mar 1995. [Online]. Available: <https://doi.org/10.1145/201059.201061>
- [31] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Efficiently computing static single assignment form and the control dependence graph,” *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, p. 451–490, Oct. 1991. [Online]. Available: <https://doi.org/10.1145/115372.115370>

- [32] F. Rastello and F. B. Tichadou, *SSA-based Compiler Design*. Springer, 2022. [Online]. Available: <https://doi.org/10.1007/978-3-030-80515-9>
- [33] F. Niephaus, T. Felgentreff, and R. Hirschfeld, “Graalsqueak: A fast smalltalk bytecode interpreter written in an ast interpreter framework,” in *Proceedings of the 13th Workshop on Implementation, Compilation, Optimization of Object-Oriented Languages, Programs and Systems*, ser. ICIOOLPS ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 30–35. [Online]. Available: <https://doi.org/10.1145/3242947.3242948>
- [34] “Graalwasm,” 2025. [Online]. Available: <https://www.graalvm.org/webassembly/>
- [35] A. Haas, A. Rossberg, D. L. Schuff, B. L. Titzer, M. Holman, D. Gohman, L. Wagner, A. Zakai, and J. Bastien, “Bringing the web up to speed with webassembly,” in *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, ser. PLDI 2017. New York, NY, USA: Association for Computing Machinery, 2017, p. 185–200. [Online]. Available: <https://doi.org/10.1145/3062341.3062363>
- [36] F. Huemer, “Full wasm support for graalvm node.js applications,” 2023. [Online]. Available: <https://resolver.obvsg.at/urn:nbn:at:at-ubl:1-59191>
- [37] S. S. Muchnick, *Advanced Compiler Design and Implementation*. Morgan Kaufmann, 1997.
- [38] “Explode loop annotation,” 2025. [Online]. Available: <https://github.com/oracle/graal/blob/master/truffle/src/com.oracle.truffle.api/src/com/oracle/truffle/api/nodes/ExplodeLoop.java>
- [39] M. Das, T. Reps, and P. van Hentenryck, “Semantic foundations of binding-time analysis for imperative programs,” in *Proceedings of the 1995 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*, ser. PEPM ’95. New York, NY, USA: Association for Computing Machinery, 1995, p. 100–110. [Online]. Available: <https://doi.org/10.1145/215465.215569>
- [40] M. L. Ponisio, “A binding-time analysis for petitcafé,” Master’s thesis, Ecole des Mines de Nantes, 2002. [Online]. Available: <https://soft.vub.ac.be/Publications/2002/Ponisio2002-EMOSE.pdf>
- [41] U. Hölzle, C. Chambers, and D. Ungar, “Optimizing dynamically-typed object-oriented languages with polymorphic inline caches,” in *ECOOP’91 European Conference on Object-Oriented Programming*, P. America, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991, pp. 21–38. [Online]. Available: <https://doi.org/10.1007/BFb0057013>
- [42] D. Leopoldseder, L. Stadler, T. Würthinger, J. Eisl, D. Simon, and H. Mössenböck, “Dominance-based duplication simulation (dbds): code duplication to enable compiler optimizations,” in *Proceedings of the 2018 ACM SIGPLAN Symposium on Code Generation and Optimization*, ser. CGO ’18. New York, NY, USA: Association for Computing Machinery, 2018, p. 126–137. [Online]. Available: <https://doi.org/10.1145/3168811>
- [43] M. A. Bulyonkov, “Polyvariant mixed computation for analyzer programs,” *Acta Inf.*, vol. 21, no. 5, p. 473–484, Dec. 1984. [Online]. Available: <https://doi.org/10.1007/BF00271642>
- [44] L. Hornof, C. Consel, and J. Noyé, “Effective specialization of realistic programs via use sensitivity,” in *Static Analysis*, P. Van Hentenryck, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1997, pp. 293–314. [Online]. Available: <https://doi.org/10.1007/BFb0032749>
- [45] B. K. Rosen, M. N. Wegman, and F. K. Zadeck, “Global value numbers and redundant computations,” in *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, ser. POPL ’88. New York, NY, USA: Association for Computing Machinery, 1988, p. 12–27. [Online]. Available: <https://doi.org/10.1145/73560.73562>
- [46] T. Ball and J. Larus, “Efficient path profiling,” in *Proceedings of the 29th Annual IEEE/ACM International Symposium on Microarchitecture. MICRO 29*, 1996, pp. 46–57. [Online]. Available: <https://doi.org/10.1109/MICRO.1996.566449>
- [47] “Wasm function node,” 2025. [Online]. Available: <https://github.com/oracle/graal/blob/master/wasm/src/org.graalvm.wasm/src/org.graalvm/wasm/nodes/WasmFunctionNode.java>
- [48] S. Barati and M. Saboff, “Introducing the jetstream 2 benchmark suite,” 2019. [Online]. Available: <https://webkit.org/blog/8685/introducing-the-jetstream-2-benchmark-suite/>
- [49] M. Guthaus, J. Ringenber, D. Ernst, T. Austin, T. Mudge, and R. Brown, “Mibench: A free, commercially representative embedded benchmark suite,” in *Proceedings of the Fourth Annual IEEE International Workshop on Workload Characterization. WWC-4 (Cat. No.01EX538)*, 2001, pp. 3–14. [Online]. Available: <https://doi.org/10.1109/WWC.2001.990739>
- [50] J. Moffitt, “Ogg vorbis—open, free audio—set your media free,” *Linux J.*, vol. 2001, no. 81es, p. 9–es, Jan. 2001. [Online]. Available: <https://dl.acm.org/doi/10.5555/364682.364691>
- [51] S. O’Dwyer, “Image processing with webassembly,” 2020. [Online]. Available: <https://silvia-odwyer.github.io/photof/>
- [52] “Graalwasm repository,” 2025. [Online]. Available: <https://github.com/oracle/graal/tree/master/wasm>
- [53] E. O. Brigham and R. E. Marrow, “The fast fourier transform,” *IEEE Spectrum*, vol. 4, no. 12, pp. 63–70, 1967. [Online]. Available: <https://doi.org/10.1109/MSPEC.1967.5217220>
- [54] W. S. Team, “Wasi-enabled webassembly c/c++ toolchain,” 2025. [Online]. Available: <https://github.com/WebAssembly/wasi-sdk>
- [55] “Graalvm repository,” 2025. [Online]. Available: <https://github.com/oracle/graal/>
- [56] “Labs jdk,” 2025. [Online]. Available: <https://github.com/graalvm/labs-openjdk>
- [57] M. W. Browne, “Cross-validation methods,” *Journal of Mathematical Psychology*, vol. 44, no. 1, pp. 108–132, 2000. [Online]. Available: <https://doi.org/10.1006/jmps.1999.1279>
- [58] F. Huemer, A. Prokopec, D. Leopoldseder, R. Mosaner, and H. Mössenböck, “Partial-evaluation templates: Additional metrics,” 2026. [Online]. Available: <https://ssw.jku.at/General/Staff/Huemer/CGO2026/>
- [59] N. D. Jones, P. Sestoft, and H. Søndergaard, “An experiment in partial evaluation: the generation of a compiler generator,” *SIGPLAN Not.*, vol. 20, no. 8, p. 82–87, Aug. 1985. [Online]. Available: <https://doi.org/10.1145/988346.988358>
- [60] C. Consel and O. Danvy, “Partial evaluation: Principles and perspectives,” in *Proceedings of the ACM-SIGPLAN Symposium on Principles of Programming Languages POPL*, vol. 93, 1993.
- [61] R. Glück and J. Jørgensen, “Efficient multi-level generating extensions for program specialization,” in *Programming Languages: Implementations, Logics and Programs*, M. Hermenegildo and S. D. Swierstra, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 1995, pp. 259–278. [Online]. Available: <https://doi.org/10.1007/BFb0026825>
- [62] P. J. Thiemann, “Cogen in six lines,” in *Proceedings of the First ACM SIGPLAN International Conference on Functional Programming*, ser. ICFP ’96. New York, NY, USA: Association for Computing Machinery, 1996, p. 180–189. [Online]. Available: <https://doi.org/10.1145/232627.232647>
- [63] C. Consel, “Polyvariant binding-time analysis for applicative languages,” in *Proceedings of the 1993 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*, ser. PEPM ’93. New York, NY, USA: Association for Computing Machinery, 1993, p. 66–77. [Online]. Available: <https://doi.org/10.1145/154630.154638>
- [64] C. Consel and O. Danvy, “For a better support of static data flow,” in *Functional Programming Languages and Computer Architecture*, J. Hughes, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 1991, pp. 496–519. [Online]. Available: https://doi.org/10.1007/3540543961_24
- [65] B. Grant, M. Mock, M. Philipose, C. Chambers, and S. J. Eggers, “Dyc: an expressive annotation-directed dynamic compiler for c,” *Theoretical Computer Science*, vol. 248, no. 1, pp. 147–199, 2000, pEPM’97. [Online]. Available: [https://doi.org/10.1016/S0304-3975\(00\)00051-7](https://doi.org/10.1016/S0304-3975(00)00051-7)
- [66] F. Noel, L. Hornof, C. Consel, and J. Lawall, “Automatic, template-based run-time specialization: implementation and experimental study,” in *Proceedings of the 1998 International Conference on Computer Languages (Cat. No.98CB36225)*, 1998, pp. 132–142. [Online]. Available: <https://doi.org/10.1109/ICCL.1998.674164>
- [67] F. Smith, D. Grossman, G. Morrisett, L. Hornof, and T. Jim, “Compiling for template-based run-time code generation,” *J. Funct. Program.*, vol. 13, no. 3, p. 677–708, May 2003. [Online]. Available: <https://doi.org/10.1017/S095679680200463X>
- [68] D. Simon, C. Wimmer, B. Urban, G. Duboscq, L. Stadler, and T. Würthinger, “Snippets: Taking the high road to a low level,” *ACM Trans. Archit. Code Optim.*, vol. 12, no. 2, Jun. 2015. [Online]. Available: <https://doi.org/10.1145/2764907>
- [69] S. Thibault, C. Consel, J. L. Lawall, R. Marlet, and G. Muller, “Static and dynamic program compilation by interpreter specialization,” *Higher-Order and Symbolic Computation*, vol. 13, pp. 161–178, 2000. [Online]. Available: <https://doi.org/10.1023/A:1010078412711>
- [70] C. Wimmer, M. Haupt, M. L. Van De Vanter, M. Jordan, L. Daynès, and D. Simon, “Maxine: An approachable virtual machine for, and in, java,” *ACM Trans. Archit. Code Optim.*, vol. 9, no. 4, Jan. 2013. [Online]. Available: <https://doi.org/10.1145/2400682.2400689>

- [71] H. Xu and F. Kjolstad, "Copy-and-patch compilation: a fast compilation algorithm for high-level languages and bytecode," *Proc. ACM Program. Lang.*, vol. 5, no. OOPSLA, Oct. 2021. [Online]. Available: <https://doi.org/10.1145/3485513>
- [72] P. Zhao and J. Amaral, "Function outlining and partial inlining," in *17th International Symposium on Computer Architecture and High Performance Computing (SBAC-PAD'05)*, 2005, pp. 101–108. [Online]. Available: <https://doi.org/10.1109/CAHPC.2005.26>
- [73] P. Zhao and J. N. Amaral, "Ablego: a function outlining and partial inlining framework," *Software: Practice and Experience*, vol. 37, no. 5, pp. 465–491, 2007. [Online]. Available: <https://doi.org/10.1002/spe.774>